

Relatório Técnico Detalhado — Estado Atual da Aplicação

Documento gerado a 17/09/2026 às 16:09 · Repositório GitHub (igor1fontes-Panda) · Produção:
<https://project-orgt3.vercel.app>

Este relatório descreve, em detalhe, o código atual da aplicação, o processo de build na íntegra, todas as alterações feitas no repositório GitHub nas últimas 48 horas (desde ontem até ao momento exato de agora) e os passos dados, com verificação de cada etapa.

Stack	Vite 8 + React 18 + TypeScript (strict)
Base de dados	Supabase (Postgres + RLS + Realtime)
AI providers	Google Gemini (Interactions API) → Blackbox AI (fallback)
Hospedagem	Vercel (produção) + Freebuff (sandbox/preview)
CI/CD	GitHub Actions — 6 workflows
Agentes IA	6 autónomos (Node .mjs) + 1 Insight Engine (Python)
Testes	26/26 a passar (Vitest) · Typecheck limpo · Build OK

1. Arquitetura do código — como a app está construída

A aplicação é um SPA (Single Page Application) estática construída com **Vite 8**, **React 18** e **TypeScript em modo strict**. Todo o estado do negócio vive no Supabase (Postgres), com segurança ao nível da linha (RLS), e chega ao browser em tempo real através de canais Realtime. Não existe modo demo nem dados simulados: cada número mostrado provém de uma linha real na base de dados.

Camada	Ficheiros	Linhas	Responsabilidade
Páginas (React)	src/pages/ (Landing, Auth, Dashboard)	763	UI das 3 rotas; Dashboard tem 9 módulos com menu próprio
Componentes	src/components/ (7 ficheiros)	—	Header (seletor PT/EN), LeadForm público, PaymentDetails, PunkTicker, S
Motor de negócio	src/lib/engine.ts	522	Funções puras e testadas: métricas, funil, scoring, estado dos agentes, Ac
Camada de dados	src/lib/data.ts	275	Cliente Supabase (RLS), CRUD de leads/clients/orders, memória de agent
Internacionalização	src/lib/i18n.ts	1 074	Dicionário PT (fonte da verdade) + EN, persistido em localStorage, sem de
Tipos	src/types.ts	—	Tipos de domínio (Lead, Client, Order, Activity, Metric...)
Estilo	src/index.css + tailwind.config.js	—	Tema neon/anime dourado, tokens CSS, fundo animado com prefers-redu

1.1 Os 9 módulos do Dashboard

- **Visão geral** — pulso do dia (leads, vendas, cobranças, execuções dos bots), checklist real de ativação, gráfico de 14 dias, atividade e últimas encomendas.
- **Packs de Produto** — pipeline em 5 fases (brief → evidência → montagem → revisão → pronto) com evidência verificada e portões de risco que bloqueiam avanço sem dados reais.
- **Leads** — tabela completa com scoring visual (hot/warm/cold), ação de IA e alteração de estado em otimista com rollback.
- **Análise** — receita diária (pago vs pendente), funil do pipeline, MRR por plano, receitas por método e por cliente.
- **Clientes** — criação de clientes com planos reais (Starter 1.250 Kz, Professional 2.916 Kz, Enterprise 8.333 Kz).
- **Encomendas** — criação de vendas, contas oficiais de recebimento (USD/EUR) e marcação de pagamento.
- **Agentes IA** — cartões dos 6 agentes com memória real + submenu **Academia IA** (currículo, briefings de mercado, formatura).
- **Operações** — missões do Gestor IA, garantia de qualidade de entregas e skills do ecossistema skills.sh.
- **Ecossistema** — ciclo autónomo de 5 etapas, estado Realtime, canais autorizados e memória aprendida.

1.2 Segurança (RLS)

- Utilizadores autenticados: acesso completo ao negócio; anónimos: apenas inserir leads (formulário público), nunca ler.
- A chave publishable (pública) vive no browser — é segura porque a RLS é aplicada no servidor.
- A service-role key vive apenas em segredos do GitHub / ambiente do servidor; nunca chega ao browser.

2. A equipa de agentes IA (7 agentes autónomos)

Todos os agentes correm no GitHub Actions, operam apenas sobre dados reais, comunicam entre si através da tabela `agent_memory` e registam tudo no `activity_log`. Se uma chave de IA faltar, degradam graciosamente para heurísticas determinísticas — nunca simulam.

Agente	Ficheiro	Cadência	Função e modus operandi
AI Manager (chefe)	scripts/ops-manager.mjs	Diária 05:30 UTC	Lê dados reais e atribui missões diárias a cada agente; regista automaticamente
Skills Scout	scripts/skills-scout.mjs	Diária 05:30 UTC	Liga a equipa ao skills.sh: pesquisa skills reais por área (via <code>/api/search</code>), b
Lead Qualifier	scripts/lead-hunter.mjs	Diária 06:00 UTC	Qualifica leads reais 0-100 (missão + briefing + viés de canal aprendido); p
Growth & Marketing	scripts/growth-marketing.mjs	Diária 06:00 UTC	Deteta o gargalo do funil e produz UM plano de campanha concreto; compo
AI Teacher	scripts/teacher-agent.mjs	Diária 06:00 UTC	Academia IA: estuda o mercado real, diagnostica lacunas e escreve um ma
Error Handler + QA	scripts/error-handler.mjs	Horária	Triagem de incidentes reais com memória de correções + QA de entregas:
Insight Engine	ai_engine.py	Diária 06:00 UTC	Analista com memória: insight baseado em métricas reais + briefing de voz

Ciclo autónomo diário: **05:30** Gestor IA distribui missões e o Olheiro de Skills traz técnicas novas do skills.sh → **06:00** os agentes executam (qualificar, campanhas, aulas, insights) → **horária** QA de entregas e triagem. Tudo auditável no módulo Operações e no feed de atividade.

3. Base de dados (Supabase) — 5 migrações idempotentes

Migração	Conteúdo	Estado
0001_init.sql (158 linhas)	Tabelas leads, clients, orders, activity_log + RLS completa + política pública de inspeção de leads + índices	Aplicada
0002_agent_memory.sql (41)	Tabela agent_memory (upsert por agente/chave) — a memória partilhada de aprendizagem de todos os agentes	Aplicada
0003_stripe_orders.sql (3)	Coluna de integração Stripe nas encomendas	Aplicada
0004_admin_rls.sql (26)	Reforço de políticas admin	Aplicada
0005_delivery_qa.sql (35)	Tabela delivery_status (QA de entregas: pack, cliente, verificações JSONB, verificação de segurança) + Políticas de índices	Não aplicada

As migrações 0001-0004 são idempotentes (if not exists, drop policy if exists, blocos DO guardados), pelo que o workflow automático de db push pode reexecutá-las em segurança. A 0005 será aplicada automaticamente no próximo merge para main (secret SUPABASE_ACCESS_TOKEN) ou manualmente no SQL Editor.

4. O build na íntegra — do código ao produção

4.1 Build local/CI (npm run build → vite build)

- Rolldown (bundler do Vite 8) resolve os ~2 537 módulos a partir de index.html → src/main.tsx → App.tsx.
- plugin-react transforma JSX/TSX; TypeScript já validou os tipos previamente (tsc -b --noEmit em CI).
- Tailwind + PostCSS + Autoprefixer processam o CSS (33 KB, 7,6 KB gzip) com os tokens do tema.
- Code-splitting desativado propositalmente (manualChunks: undefined): a config pública do Supabase vive num único chunk principal, para qualquer CDN servir a app só com index.html.
- Recharts, framer-motion, lucide-react e @supabase/supabase-js entram no bundle (1 063 KB, 308 KB gzip).
- Saída: dist/ (index.html + assets/ com hash imutável, cache de 1 ano em vercel.json).

4.2 Pipeline de deploy (3 vias)

- **Vercel (produção, ativa):** push para main → a integração nativa da Vercel corre npm ci + npm run build → publica dist/ em https://project-orgt3.vercel.app. rewrites de SPA e headers de cache em vercel.json. Analytics de velocidade real via Speed Insights.
- **GitHub Actions (opcional):** deploy.yml corre se existirem os secrets VERCEL_TOKEN/ORG_ID/PROJECT_ID; salta limpo caso contrário.
- **Freebuff (sandbox/preview):** instalação npm ci + build com Vite para pré-visualização isolada; o Deploy button publica na hospedagem gerida.

4.3 Verificações em cada push (CI — ci.yml)

- npm run typecheck → tsc -b --noEmit (strict: zero erros atualmente).
- npx vitest run → 26 testes unitários do motor de negócio (26/26 a passar).
- npm run build → build de produção tem de compilar sem falhas.
- Verificação de fuga de segredos: nenhum SUPABASE_SECRET/SERVICE_ROLE/GEMINI/BLACKBOX no código fonte (src/ e public/).

5. O repositório GitHub — o que foi feito desde ontem

O repositório tem 69 commits; o último merge foi a 15/09 (PR #41 — correção de conflito do workflow CodeQL). Desde ontem foram feitas as seguintes entregas (working tree atual, prontas a commit pelo painel Changes):

Integração Supabase completa

- Suporte às novas chaves `sb_secret_` (SUPABASE_SECRET_KEY) em todos os bots, mantendo a legacy `SERVICE_ROLE` — ambos aceites.
- CLI Supabase configurado (supabase/config.toml cometido; estado local .temp ignorado no .gitignore).
- Workflow supabase-migrations.yml: supabase db push automático em cada merge para main que altere migrations/ (salta limpo sem o secret SUPABASE_ACCESS_TOKEN).

Limpeza e organização do repositório

- Removido supabase/.temp/cli-latest do índice (estado local do CLI nunca devia estar cometido).
- Eliminada a migração vazia 20260913145246_new-migration.sql (tutorial do CLI) que criaria um registo no-op na história de produção.
- 5 PDFs de relatórios movidos de docs/ e da raiz para docs/reports/.
- Dependências não usadas removidas: @supabase/server e @supabase/ssr (zero imports no código).

Interface bilingue PT/EN

- Novo src/lib/i18n.ts (1 074 linhas): PT como fonte da verdade, EN espelhado; persistência em localStorage e sincronizado.
- Seletor PT | EN no header (desktop e mobile); PT por defeito (público Angola/Portugal).
- Todos os ecrãs traduzidos: landing, auth, dashboard (9 módulos), formulários, pagamentos, footer.

Fundo anime animado + UI clara

- Fundo neon estendido ao dashboard e auth (antes só a landing); animação de drift; respeita prefers-reduced-motion.
- Menu de módulos em grelha — cada cartão abre o seu próprio conteúdo; cartões de métricas navegam para o módulo certo.

Academia IA (agentes que se ensinam a vender)

- scripts/teacher-agent.mjs: Professor IA estuda o mercado real, diagnostica lacunas por agente e escreve um market_brief individual na memória de cada aluno.
- Consumo automático do briefing pelo Lead Qualifier e pelo Growth & Marketing em cada run.
- Submenu Agentes | Academia IA: currículo por agente (skills provadas por memória real), formatura honesta (Inscrito → Em formação → Formado) e atualidade do briefing.
- Funções puras academyState()/briefFreshness() + 4 testes novos.

Operações — Gestor IA + skills.sh + QA

- scripts/ops-manager.mjs: missões diárias atribuídas a cada agente; registo automático de packs vendidos (pedidos pagos → delivery_status) e auto-registo de clientes novos com plano inferido pelo valor pago.
- scripts/skills-scout.mjs: integração real com skills.sh (busca semântica + npx skills use para obter instruções) e entrega de skill_entry por agente.
- QA de entregas no error-handler: 4 verificações reais por pack (pago, cliente, valor, produto operacional via SITE_URL).
- Migração 0005_delivery_qa.sql + módulo Operações no dashboard + funções opsState()/skillFreshness() + 3 testes novos.

Qualidade e verificação contínua

- TypeScript strict sem erros; 26/26 testes a passar; build de produção compilou com sucesso após cada etapa.
- Sintaxe validada nos 6 bots Node; YAML dos 6 workflows validado; npm ls confirmou árvore de dependências deduplicada.

Ficheiros atualmente modificados/criados (working tree): 30 ficheiros, +2 024 / -640 linhas. Novos: scripts/ops-manager.mjs, scripts/skills-scout.mjs, scripts/teacher-agent.mjs, scripts/growth-marketing.mjs, src/lib/i18n.ts, supabase/migrations/0005_delivery_qa.sql, supabase/config.toml, .github/workflows/supabase-migrations.yml.

6. Estado atual exato e próximos passos

Item	Estado agora
Typecheck (tsc -b strict)	■ Sem erros
Testes unitários (Vitest)	■ 26/26 a passar
Build de produção (vite build)	■ dist/ gerado (1 063 KB → 308 KB gzip)
Bots (6 Node + 1 Python)	■ Sintaxe validada
Workflows (6 YAML)	■ Válidos
Produção (Vercel)	■ https://project-orgt3.vercel.app a servir a build anterior
Working tree	30 ficheiros alterados, aguardam commit pelo painel Changes

6.1 Ações que só tu podes fazer (segredos/consentimento)

- **Secret SUPABASE_ACCESS_TOKEN** (GitHub → Settings → Secrets → Actions): ativa o db push automático; sem ele o workflow salta limpo. O token sbp_... que colaste anteriormente é exatamente isto.
- **Secret SUPABASE_SECRET_KEY** real (o valor your-secret-key do teu paste era placeholder): recebe a sb_secret_... completa e coloca-a nos secrets do GitHub e nas env vars da Vercel.
- **Repos secreta GEMINI_API_KEY** (gratuita em aistudio.google.com) e opcionalmente BLACKBOX_API_KEY — sem elas os bots correm em modo determinístico.
- **Variable SITE_URL** (repo variable, não secreta): ativa o health check de produto na QA de entregas.
- **Migração 0005**: aplica-se automaticamente no próximo merge para main (com o ACCESS_TOKEN) ou manualmente no SQL Editor.

6.2 O que acontece automaticamente após o próximo push para main

- CI valida types + testes + build e verifica fuga de segredos.
- Vercel reconstrói e publica a nova versão em produção.
- supabase-migrations.yml aplica a migração 0005 (se o ACCESS_TOKEN existir).
- Às 05:30 UTC: Gestor IA distribui missões + Skills Scout traz skills do skills.sh; às 06:00: agentes executam; de hora a hora: QA de entregas.

Fontes AI Admin Adjunta Solutions · Documento gerado automaticamente a partir do estado real do repositório · 17/09/2026